PART B

Experiment no: - 1

Experiment Name: - DDL statements.

Aim: - To perform practicals using DDL statements for creating tables, managing table structures, and applying constraints in the Cinema Ticket Booking System.

Roll. No: C-60	Name: Avdhut Madhavi
Class: COMP-C	Batch: C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a SQL statement and check the output. (Put the query and its output after each question.)

1. Create MOVIE table

```
CREATE TABLE MOVIE (
   movie_id INT PRIMARY KEY,
   title VARCHAR(50) NOT NULL,
   genre VARCHAR(20),
   language VARCHAR(20),
   duration INT,
   rating DECIMAL(3,1)
      CHECK (rating BETWEEN 0 AND 10)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
movie_id	int	NO	PRI	NULL	
title	varchar(50)	NO		NULL	
genre	varchar(20)	YES		NULL	
language	varchar(20)	YES		NULL	
duration	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	

CREATE TABLE THEATRE (
theatre_id NUMBER(5) PRIMARY KEY,
theatre_name VARCHAR2(50) NOT NULL,
city VARCHAR2(30),
phone VARCHAR2(15) UNIQUE);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
theatre_id	int	NO	PRI	NULL	
theatre_name	varchar(50)	NO		NULL	
city	varchar(30)	YES		NULL	
phone	varchar(15)	YES	UNI	NULL	

```
CREATE TABLE SCREEN (
   screen_id INT PRIMARY KEY,
   screen_name VARCHAR(20) NOT NULL,
   capacity INT CHECK (capacity > 0),
   theatre_id INT,
   FOREIGN KEY (theatre_id)
      REFERENCES THEATRE(theatre_id)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

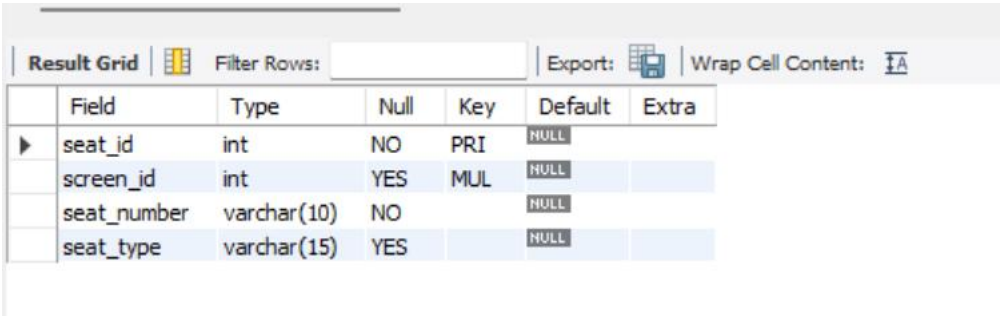
Field	Type	Null	Key	Default	Extra
screen_id	int	NO	PRI	NULL	
screen_name	varchar(20)	NO		NULL	
capacity	int	YES		NULL	
theatre_id	int	YES	MUL	NULL	

```
CREATE TABLE SHOWTIME (
   show_id INT PRIMARY KEY,
   movie_id INT,
   screen_id INT,
   show_date DATE,
   show_time TIME,
   ticket_price DECIMAL(8,2)
      CHECK (ticket_price > 0),
   FOREIGN KEY (movie_id)
      REFERENCES MOVIE(movie_id),
   FOREIGN KEY (screen_id)
      REFERENCES SCREEN(screen_id)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
show_id	int	NO	PRI	NULL	
movie_id	int	YES	MUL	NULL	
screen_id	int	YES	MUL	NULL	
show_date	date	YES		NULL	
show_time	time	YES		NULL	
ticket_price	decimal(8,2)	YES		NULL	

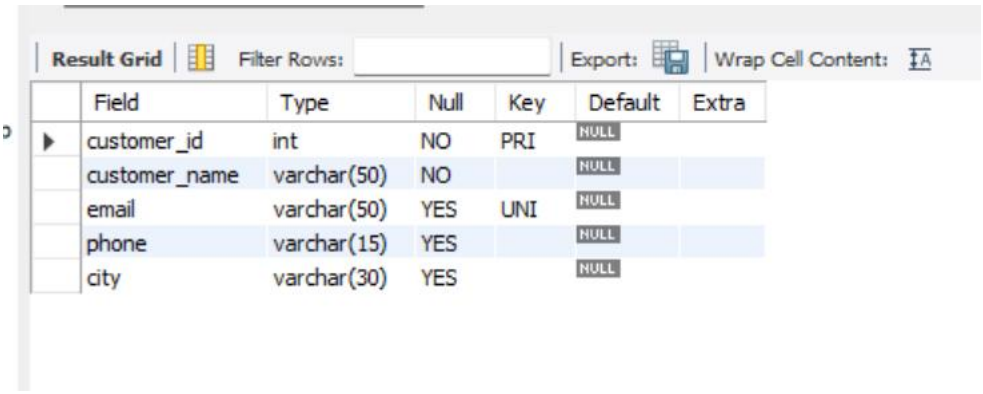
```
CREATE TABLE SEAT (
   seat_id INT PRIMARY KEY,
   screen_id INT,
   seat_number VARCHAR(10) NOT NULL,
   seat_type VARCHAR(15),
   UNIQUE (screen_id, seat_number),
   FOREIGN KEY (screen_id)
      REFERENCES SCREEN(screen_id)
);
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
seat_id	int	NO	PRI	NULL	
screen_id	int	YES	MUL	NULL	
seat_number	varchar(10)	NO		NULL	
seat_type	varchar(15)	YES		NULL	

```
CREATE TABLE CUSTOMER (
   customer_id INT PRIMARY KEY,
   customer_name VARCHAR(50) NOT NULL,
   email VARCHAR(50) UNIQUE,
   phone VARCHAR(15),
   city VARCHAR(30)
);
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
customer_name	varchar(50)	NO		NULL	
email	varchar(50)	YES	UNI	NULL	
phone	varchar(15)	YES		NULL	
city	varchar(30)	YES		NULL	

ALTER – Modify table structure

A. Add a column
Add a remarks column to Scheme_Details.

```
ALTER TABLE MOVIE
ADD remarks VARCHAR(100);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
movie_id	int	NO	PRI	NULL	
title	varchar(50)	NO		NULL	
genre	varchar(20)	YES		NULL	
language	varchar(20)	YES		NULL	
duration	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	
remarks	varchar(100)	YES		NULL	

B. Modify a column
Increase the size of the title column from 50 to 100 characters.

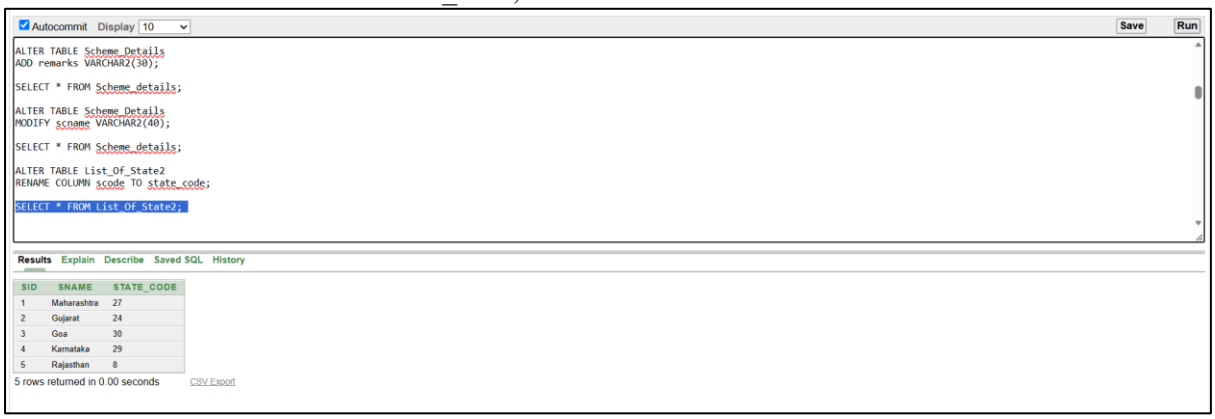
```
ALTER TABLE MOVIE
MODIFY COLUMN title VARCHAR(100);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
movie_id	int	NO	PRI	NULL	
title	varchar(100)	YES		NULL	
genre	varchar(20)	YES		NULL	
language	varchar(20)	YES		NULL	
duration	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	
remarks	varchar(100)	YES		NULL	

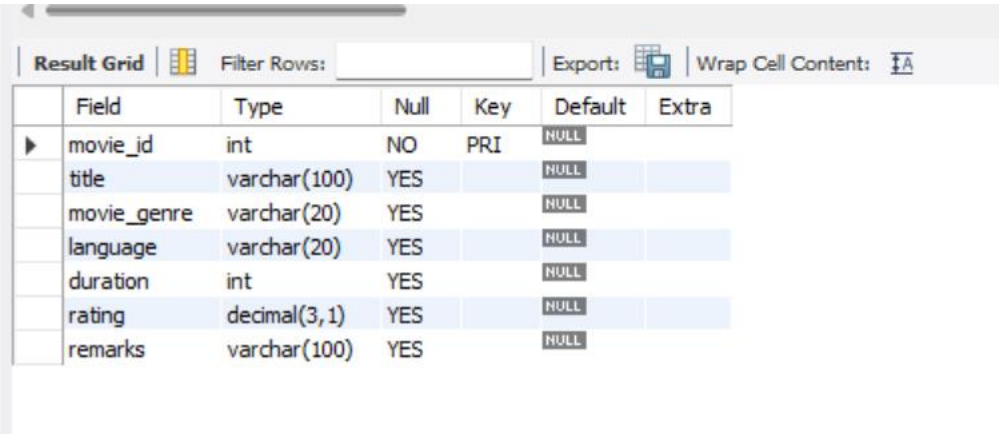
C. Rename a column
Rename the genre column to movie_genre.

```
ALTER TABLE List_Of_State2
RENAME COLUMN scode TO state_code;
```



D. Drop a column
Remove the remarks column from Scheme_Details.

```
ALTER TABLE Scheme_Details
DROP COLUMN remarks;
```

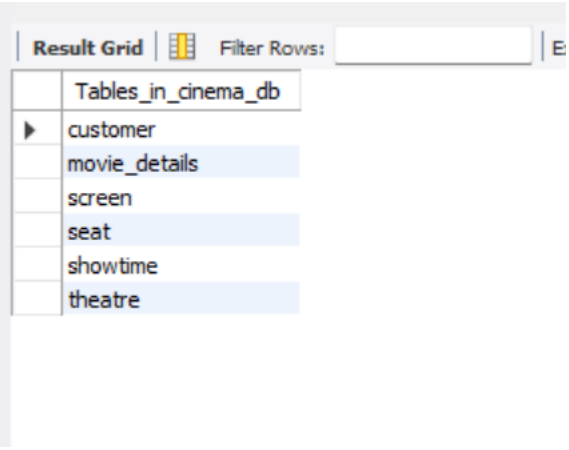


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
movie_id	int	NO	PRI	NULL	
title	varchar(100)	YES		NULL	
movie_genre	varchar(20)	YES		NULL	
language	varchar(20)	YES		NULL	
duration	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	
remarks	varchar(100)	YES		NULL	

3. RENAME – Rename a table
Rename MOVIE to MOVIE_DETAILS.

RENAME TABLE MOVIE TO MOVIE_DETAILS;



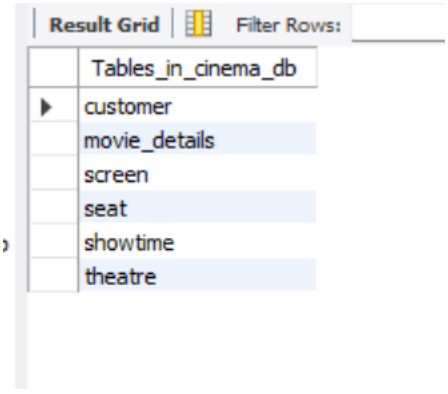
Result Grid | Filter Rows:

Tables_in_cinema_db
customer
movie_details
screen
seat
showtime
theatre

4.DROP – Delete a table

This sql query deletes the table which we created just for this querry.

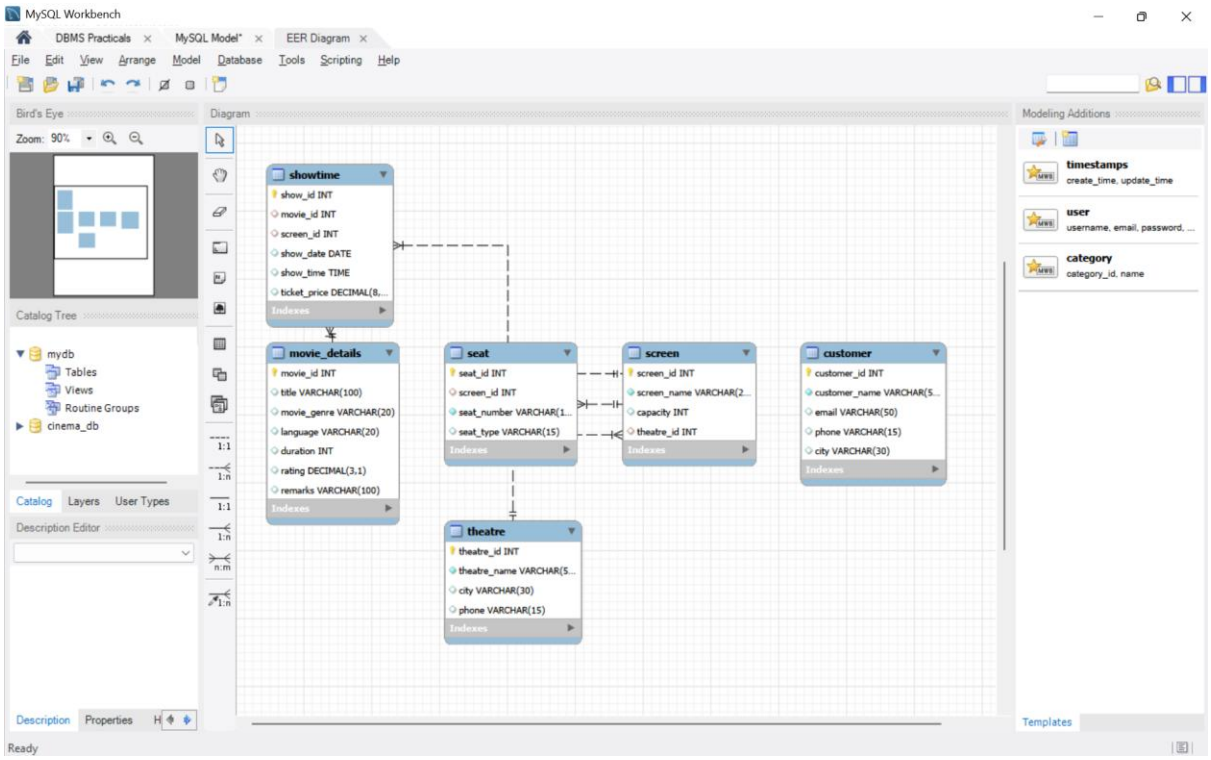
DROP TABLE TEST_RENAME_NEW;



Result Grid | Filter Rows:

Tables_in_cinema_db
customer
movie_details
screen
seat
showtime
theatre

EER Diagram:



B.2: Write an observation according to the query execution

1. The CREATE command was used to create tables in the Cinema Ticket Booking System.
2. The ALTER command was used to add, modify, rename, and drop columns.
3. The RENAME TABLE command was used to change the name of an existing table.
4. The TRUNCATE command was used to delete all records while retaining the table structure.
5. The DROP TABLE command was used to delete a table along with its structure and data.
6. DDL commands helped us understand how to create and manage database structures.

B.3: Write an answer of the following questions

1. What is the difference between the CREATE and ALTER statements?

CREATE	ALTER
1. It is used to create a new table.	1. It is used to modify an existing table.
2. It defines the table structure.	2. It changes the table structure.
3. It creates a new database object.	3. It modifies an existing database object.
Example: CREATE TABLE Student (id NUMBER);	Example: ALTER TABLE Student ADD name VARCHAR2(20);

2. Explain the difference between DROP and TRUNCATE statements.

DROP	TRUNCATE
1. It deletes the entire table.	1. It deletes all records from a table.
2. It removes both data and table structure.	2. It keeps the table structure.
3. The table cannot be used until it is created again.	3. New records can be inserted into the table.
Example: DROP TABLE Student;	Example: TRUNCATE TABLE Student;

3. What happens when you use the DROP TABLE command?

When we use the DROP TABLE command:

1. The entire table is deleted from the database.
2. All records stored in the table are removed.
3. The table structure, including its columns, is deleted.
4. Related constraints and indexes associated with the table are also removed.
5. The table must be created again if we want to use it in the future.

EXAMPLE :-

DROP TABLE Student;